INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Excelencia en Educación Superior

1.1 Neurona Perceptron

Erick Mejia

Caso Práctico: Neurona de McCulloch y Pitts

2. Aplicando la MPNeurona a un caso práctico real

2.1. Conjunto de datos

Esta es una copia del conjunto de datos de UCI ML Breast Cancer Wisconsin.

Descripción de la muestra

Las características de entrada se calculan a partir de una imagen digitalizada de un aspirado de aguja fina (FNA) de una masa mamaria. Describen las características de los núcleos celulares presentes en la imagen.

El plan de separación descrito anteriormente se obtuvo utilizando el método de árbol de múltiples superficies (MSM-T) "Construcción de un árbol de decisión mediante programación lineal".

El programa lineal utilizado para obtener el plano de separación en el espacio tridimensional es el que describe en [K. P. Bennett y O. L. Mangasarian].

Lectura de datos

In [1]:
```
import pandas as pd
from sklearn.datasets import load_breast_cancer

# Cargamos el dataset completo
cancer_mama = load_breast_cancer()

# --- CAMBIO AQUÍ: Sacamos las 10 más importantes mediante correlación ---
# Creamos un DF temporal para calcular qué columnas mandan
df_temp = pd.DataFrame(cancer_mama.data, columns=cancer_mama.feature_names)
df_temp['target'] = cancer_mama.target

# Buscamos las 10 con mayor correlación absoluta con el target
correlations = df_temp.corr()['target'].abs().sort_values(ascending=False)
top_10_features = correlations.iloc[1:11].index.tolist()

# Definimos X solo con esas 10 y Y (usamos .values para que sea matriz)
X = df_temp[top_10_features].values
Y = cancer_mama.target

print("Top 10 características seleccionadas:\n", top_10_features)
```

```
Top 10 características seleccionadas:
 ['worst concave points', 'worst perimeter', 'mean concave points', 'worst radius',
'mean perimeter', 'worst area', 'mean radius', 'mean area', 'mean concavity', 'worst
concavity']
```

Visualización del conjunto de datos

In [2]:
```
import pandas as pd

# Usamos las feature_names filtradas
df = pd.DataFrame(X, columns=top_10_features)
```

In [3]:
```
print(Y)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

División del conjunto de datos

In [4]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(df, Y, stratify=Y, random_state

print("Tamaño de entrenamiento ", len(X_train))
print("Tamaño de pruebas", len(X_test))
```

```
Tamaño de entrenamiento  426
Tamaño de pruebas 143
```

Implementación de un aMP Neurona más avanzada

In [5]:
```python
# importamos la biblioteca de numpy
import numpy as np
from sklearn.metrics import accuracy_score

class MpNeurona:
    def __init__(self):
        self.threshold = None

    def model(self, x):
        z = sum(x)
        return (z >= self.threshold)

    def predict(self, X):
        Y = []
        for x in X:
            result = self.model(x)
            Y.append(result)
        return np.array(Y)

    def fit(self, X, Y):
        accuracy = {}
        for th in range(X.shape[1] + 1):
            self.threshold = th
```

```
            Y_pred = self.predict(X)
            accuracy[th] = accuracy_score(Y_pred, Y)
        self.threshold = max(accuracy, key=accuracy.get)
```

Los datos de entrada siguen teniendo problemas debido a que en nuestro conjunto de datos las características de entrada reciben valores continuos; sin embargo, la neurona MP solo procesa características de entrada con valores binarios.

In [6]:
```
# Transformamos las características de entrada a binario (0 y 1)
X_train_bin = X_train.apply(pd.cut, bins=2, labels=[1, 0]).astype(int)
X_test_bin = X_test.apply(pd.cut, bins=2, labels=[1, 0]).astype(int)

# Instanciamos y entrenamos
np_neurona = MpNeurona()
np_neurona.fit(X_train_bin.to_numpy(), y_train)

print("Threshold optimo seleccionado:", np_neurona.threshold)

# Validación rápida
Y_pred = np_neurona.predict(X_test_bin.to_numpy())
print("Exactitud en test:", accuracy_score(y_test, Y_pred))
```

```
Threshold optimo seleccionado: 10
Exactitud en test: 0.9090909090909091
```

In [ ]:
```
print("--- INGRESO DE DATOS MANUALES (VALORES CONTINUOS) ---")
datos_usuario_continuos = []

# Pedir los 10 datos por teclado
for col in top_10_features:
    valor = float(input(f"Ingrese valor para {col}: "))
    datos_usuario_continuos.append(valor)

# Transformar a binario comparando con el promedio del entrenamiento
# (Si el valor es menor al promedio, es 1, si es mayor es 0, según tu lógica de pd.
datos_usuario_binarios = []
for i, valor in enumerate(datos_usuario_continuos):
    promedio = X_train.iloc[:, i].mean()
    datos_usuario_binarios.append(1 if valor < promedio else 0)

# Verificar con la neurona
resultado = np_neurona.model(datos_usuario_binarios)

print("\n--- RESULTADO ---")
print("Vector binario generado:", datos_usuario_binarios)
if resultado:
    print("El paciente NO tiene cáncer (Benigno)")
else:
    print("El paciente TIENE cáncer (Maligno)")
```

```
--- INGRESO DE DATOS MANUALES (VALORES CONTINUOS) ---
```

9 Github

https://github.com/Erick-305/machine-learning.git